

Notes on PPO, DPO, and GRPO

Here are comprehensive notes on PPO, DPO, and GRPO. These notes cover the conceptual framework, the mathematical equations, and the comparative differences and advantages of each method.

Part 1: The Context of Reinforcement Learning (RL) in LLMs

- **The pipeline:** LLM training often moves from pre-training (predicting the next token) $\rightarrow$ supervised fine-tuning (instruction tuning) $\rightarrow$ preference/reasoning fine-tuning.
- **The goal:** Once internet-scale text data is exhausted, models can improve using implicit signals such as human preferences (choosing between two answers) or verifiable rewards (e.g., math/code correctness).
- **The challenge:** We want to update the model (the *policy*) from a single final score (a *reward*), back-propagating that signal through a long trajectory of tokens. This can be difficult due to high variance (noisy data causing unstable updates).

Part 2: Proximal Policy Optimization (PPO)

The standard, robust actor–critic approach

Core concept. PPO is an actor–critic method using two distinct learned components during training:

1. **Actor (policy π_θ):** the LLM generating text.
2. **Critic (value function V_ϕ):** estimates the value of a state (expected future reward from this point).

The math and equations.

1. **Advantage function (A_t).** PPO determines whether an action was “good” by computing the advantage:

$$A_t = Q(s_t, a_t) - V(s_t).$$

Here, $Q(s_t, a_t)$ is the (realized) value/reward obtained from the action and $V(s_t)$ is the baseline expected value estimated by the critic. Intuitively, if you score 10 points (Q) but the critic expected 5 (V), then $A_t = +5$.

2. **Clipping mechanism (the “proximal” part).** To prevent overly large updates from noisy data, PPO clips the policy update using a trust-region-like constraint:

$$L^{\text{CLIP}} = E \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right],$$

where the probability ratio is

$$r_t(\theta) = \frac{\pi_{\text{new}}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}.$$

The hyperparameter ϵ (often 0.1 or 0.2) defines the clipping range; if the ratio becomes too large (e.g. above $1 + \epsilon$), the objective is clipped to keep π_{new} close to π_{old} .

Advantages & disadvantages.

- **Advantage:** very stable due to clipping; helps prevent “model collapse” (catastrophic drift).
- **Disadvantage:** high compute and memory cost; commonly involves multiple models/components (policy, reference, reward model, and value function/critic).

Part 3: Direct Preference Optimization (DPO)

The efficient, non-RL approach

Core concept. DPO bypasses actor–critic RL by turning preference tuning into a direct optimization problem (often viewed as a classification-style objective) with a loss that implicitly captures reward differences.

The math and equations.

1. **Bradley–Terry preference model.** DPO relates preference probabilities to reward differences:

$$P(y_w > y_l) = \sigma(r(x, y_w) - r(x, y_l)),$$

where σ is the sigmoid, y_w is the winning response, and y_l is the losing response.

2. **DPO loss function.** By substitution, DPO removes the explicit reward model and uses log-probability ratios between the current policy and a frozen reference policy:

$$\mathcal{L}_{\text{DPO}} = -E \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right].$$

Here, π_{θ} is the model being trained, π_{ref} is the frozen reference model, and β controls how strongly the update is regularized toward the reference (often analogous to a KL weight).

Advantages & disadvantages.

- **Advantage:** efficient and simpler to implement than PPO; typically requires only the policy and a reference model (no critic, no learned reward model).

- **Disadvantage:** naturally fits pairwise preference data (winner/loser), which can be less direct for absolute, verifiable grading compared to RL-style optimization.

Part 4: Group Relative Policy Optimization (GRPO)

An efficient RL approach using group statistics (popularized for reasoning)

Core concept. GRPO is an RL algorithm like PPO but removes the value function (critic). Instead of learning a baseline with a neural network, it computes a baseline from statistics over a group of samples for the same prompt.

The math and equations.

1. **Group sampling and baseline.** For a given prompt, GRPO samples a group of outputs (e.g. G outputs $\{o_1, o_2, \dots, o_G\}$) and computes the mean and standard deviation of their rewards.
2. **Normalized advantage.** Instead of $A = \text{reward} - \text{value}$, GRPO uses a group-relative advantage:

$$A_i = \frac{r_i - \text{mean}(\text{Rewards}_{\text{group}})}{\text{std}(\text{Rewards}_{\text{group}})}.$$

Intuitively, if an output scores above the average of its sibling samples, it receives positive advantage; if below average, negative advantage.

3. **GRPO loss.** The objective resembles PPO’s clipped policy-gradient objective but typically adds an explicit KL term (since there is no critic):

$$L_{\text{GRPO}} = L_{\text{PPO_CLIP}} + \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}).$$

The clipping constrains updates, and the KL term discourages drifting too far from the reference policy π_{ref} .

Advantages & disadvantages.

- **Advantage:** major memory savings by removing the critic (often roughly halving memory/compute vs. PPO in practice); effective for reasoning tasks with verifiable rewards (math/code).
- **Disadvantage:** depends on group size; requires generating multiple outputs per prompt during training.

Summary of Differences

Feature	PPO	DPO	GRPO
Type	RL (actor-critic)	Direct preference optimization	RL (policy gradient)
Models required	Policy, reference, reward, value (critic)	Policy, reference	Policy, reference (no critic)
Baseline method	Learned value function (critic)	N/A (pairwise comparison)	Group mean/std statistics
Key mechanism	Clipping (trust region)	Log-sigmoid margin loss	Group-normalized advantage + clip + KL
Complexity	High	Low	Medium
Primary use case	General RLHF	Preference fine-tuning	Reasoning (math/code/logic)

Key takeaways on “why different?”

- **PPO vs. DPO:** DPO shows that, for preference data, you can optimize directly with a derived loss (no separate learned reward model and no critic), making it simpler and often more stable in practice.
- **PPO vs. GRPO:** GRPO removes the critic and replaces it with group statistics, cutting memory/compute while maintaining strong performance on verifiable-reward reasoning tasks.

Part 5: Policy Gradient Methods and REINFORCE

1. Policy gradient methods (the concept). Policy gradient methods are a family of reinforcement learning algorithms that optimize the **policy** (the neural network / LLM) directly.

- **Direct optimization:** Unlike value-based methods (e.g., Q-learning) that first estimate values and then derive a policy, policy gradient methods directly parameterize and optimize action probabilities (tokens).
- **Goal:** Find parameters θ that maximize expected return, typically via gradient ascent (moving “uphill” on the reward objective).

2. The REINFORCE algorithm. REINFORCE (1992) is the foundational Monte Carlo policy gradient algorithm. It updates only after an entire episode (e.g., a full conversation or reasoning chain) completes:

1. **Sample a trajectory:** The policy generates a sequence of actions $(a_1, a_2, \dots)$ and receives a final cumulative reward/return G_t .
2. **Compute returns:** After the episode ends, compute the (typically discounted) return for each time step:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots,$$

which summarizes “how good the future turned out” after taking action a_t .

3. **Update:** If the return is high, increase the probability of the actions taken; if low, decrease them.

3. The update equation (explained). The classic update rule is

$$\theta_{t+1} = \theta_t + \alpha \cdot \nabla \log \pi_{\theta}(a_t \mid s_t) \cdot G_t.$$

- α is the learning rate (step size).
- $\pi_{\theta}(a_t \mid s_t)$ is the probability of action a_t given state/context s_t under the current policy.
- $\nabla \log \pi_{\theta}(a_t \mid s_t)$ is the gradient direction that makes action a_t more likely (the log is used for convenient differentiation).
- G_t is the return (sum of future rewards from time t onward), scaling the update: positive G_t reinforces the action; negative G_t suppresses it.

Implementation as a loss (minimization form). In practice, since optimizers typically *minimize* a loss, REINFORCE is commonly implemented by minimizing the negative log-probability weighted by return:

$$\mathcal{L}_{\text{REINFORCE}} = - \sum_t G_t \ln \pi_{\theta}(a_t \mid s_t).$$

Baseline to reduce variance (common fix). Raw REINFORCE can have high variance because a delayed positive reward can reinforce *all* earlier actions in the trajectory. A standard variance-reduction technique is to subtract a baseline b :

$$\theta_{t+1} = \theta_t + \alpha \cdot \nabla \log \pi_{\theta}(a_t \mid s_t) \cdot (G_t - b),$$

which changes the signal from “good” to “better than usual.” PPO typically uses a learned value function as the baseline, while GRPO uses group statistics.

Implementation as a loss (minimization form). In practice, since optimizers typically *minimize* a loss, REINFORCE is commonly implemented by minimizing the negative log-probability weighted by return:

$$\mathcal{L}_{\text{REINFORCE}} = - \sum_t G_t \ln \pi_{\theta}(a_t \mid s_t).$$

Baseline to reduce variance (common fix). Raw REINFORCE can have high variance because a delayed positive reward can reinforce *all* earlier actions in the trajectory. A standard variance-reduction technique is to subtract a baseline b :

$$\theta_{t+1} = \theta_t + \alpha \cdot \nabla \log \pi_{\theta}(a_t \mid s_t) \cdot (G_t - b),$$

which changes the signal from “good” to “better than usual.” PPO typically uses a learned value function as the baseline, while GRPO uses group statistics.

4. Advantages and disadvantages.

Advantages	Disadvantages
Smooth probability updates (no abrupt arg max switching).	High variance: a single “lucky” high return can reinforce all actions in the trajectory, including suboptimal ones.
Naturally supports stochastic policies (can learn calibrated randomness).	Monte Carlo inefficiency: must wait for the whole episode to finish before updating.
Handles large action spaces without enumerating all actions each step.	Global reinforcement: without a baseline, if all rewards are positive it reinforces everything, which can slow learning.

Connection to PPO and GRPO. High variance is one reason PPO adds a critic/value function to stabilize the return signal, while GRPO replaces the learned baseline with group statistics to reduce variance without training a separate critic network.